Greedy Method

- Knapsack Problem :

It is a problem where :
we have a knapsack with fixed capacity w
we have n items
Each item has weight (wi), profit (pi)

Greedy method is an algorithmic strategy for solving optimization problems that makes the best possible(locally optimal) choice at each step, with the hope that these choices will lead to a globally optimal solution.

1) $W = 20$

Items	wt	profit	Pi/wi
1	14	24	24/14 = 1.7
2	18	20	20/18 = 1.1
3	10	16	16/10 = 1.6

Items	wt	profit	Pi/wi
1	14	24	1.7
3	10	16	1.6
2	18	20	1.1

Take item 1 (14) $W = 20 - 14 = 6$ profit = 24
Take item 2 (18) profit = (6/18) × 16 + 24
profit = 9.60 + 24 = 33.60

① Calculate Pi/wi
② Sort in desc
③ Calculate total profit

2) Item Profit Weight C=35

1	500	30
2	400	20
3	200	10

Step 1: Calculate Pi/Wi Ratio = Pi/Wi

Item	Profit	Ratio	Weight
1	500	16.67	30
2	400	20	20
3	200	20	10

Step 2 : Sort in descending order of Ratio

Item	Profit	Weight	Ratio
2	400	20	20
3	200	10	20
1	500	30	16.67

Step 3: Fill knapsack

Take item 2 (weight 20)

W=20 C= 35 - 20 = 15

Total profit = 400

Take item 3 (weight 10)

W=10 C= 15-10 = 5

Total profit = 400 + 200 = 600

Take item 1 (weight 30)

W=30 C= 5

Total profit = 600 + (5/30) x 500 = 8383

= 600 + 83.33 = 683.33

- Algorithm : Input n, P[], w[], C
Step 1 : For each item i
Compute ratio [i] = P[i] / w[i]
Step 2 : Sort items in decending order of ratio
Step 3 : Totalprofit = 0
Step 4 : For each item i in Sorted order
if w[i] <= C ;
Take whole item
~~total profit~~ [i] += P[i]
C = C - w[i]
else :
Take fraction = C / w[i]
total profit += P[i] * fraction
break
Step 5 : Return totalprofit

- Time complexity
1) Calculating ratios → O(n)
2) Sorting → O(n log n)
3) Filling Knapsack → O(n)

Final complexity : O(n log n)

- Job sequencing

In this we are given n jobs.
Each jobs has: deadline (di), profit (pi)
Each job take one unit and only one job can be done at a time. A job must complete before or on its deadline.

1)

Ji	Pi	di
1	60	2
2	30	1
3	40	2
4	80	1

→ Sort as per profit

Ji	Pi	di
4	80	1
1	60	2
3	40	2
2	30	1

Max dealine = 2

J4	J1
0	1 2

Total profit = 80 + 60 = 140

2)

Ji	di	Pi
1	5	200
2	3	180
3	3	190
4	2	300
5	4	120
6	2	100

Ji	di	Pi
4	2	300
1	5	200
3	3	190
2	3	180
5	4	120
6	2	100

Max dealine = 5

J2	J4	J3	J5	J1
0	1	2	3	4 5

Total profit = 300 + 200 + 190 + 120 + 180
= 990

3)

Ji	di	Pi
1	2	100
2	1	19
3	2	27
4	1	25
5	3	15

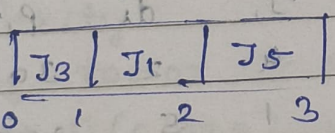
Step 1 : Sort by Profit (Descending)

Ji	di	Pi
1	2	100
3	2	27
4	1	25
2	1	19
5	3	15

Step 2 : Find maximum deadline

Max deadline = 3 , so 3 time slots

Step 3 : Assign jobs



J_1 deadline 2 , check slot 2 → empty → assign

J_3 deadline 2 , check slot 1 → empty → assign

J_4 deadline 1 , check slot 1 → cannot assign

J_2 → cannot assign

J_5 → deadline 3 , check slot 3 → assign

∴ Total profit = 100 + 27 + 15 = 142

- Algorithm steps: Input: n jobs, di array D[], P[]
  Step 1: Sort all jobs in decreasing order of profit
  Step 2: Find maximum deadline (max D)
  Step 3: Create an array slot [1.... max D],
  initialize all to -1 (empty).
  Step 4: For each job (in sorted order)
  for j = deadline down to 1
  if slot [j] is empty
  assign job to slot [j]
  break
  Step 5: Return schedule jobs and total profit.

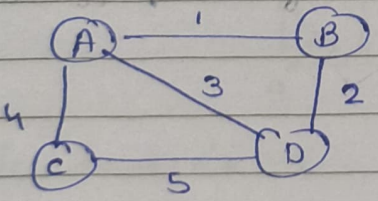
- Time complexity:
  Sorting → O(nlogn)
  Slot checking → $O(n^2)$ worst case

  Final time complexity: $O(n^2)$
  If we use disjoint set (union-find), it becomes O(nlogn)

- Spanning tree :
1) A spanning tree connects all vertices of a graph without a cycle
2) The aim of MST is to construct a tree that spans or connects all the vertices with a subset of edges with a minimum total weight

- Kruskal :
1) Input the graph
2) Sort the edges of graphs in ascending order
3) Each time select edge with min. weight
4) Include the edge in solution if it does not lead to formation of cycle else discard
5) Continue till all vertices are included :

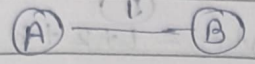
1)



Step 1 : Sort

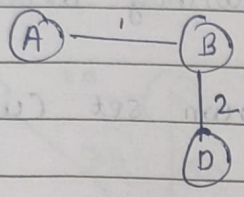
AB	1	{A} {B} {C} {D}
BD	2	
AD	3	
AC	4	
CD	5	

Step 2: select AB check for cycle using find and union
No cycle found



{AB} {C} {D}

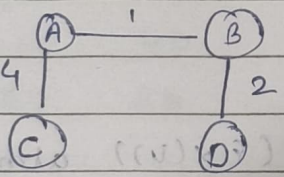
Select BD ...
No cycle found



{ABD} {C}

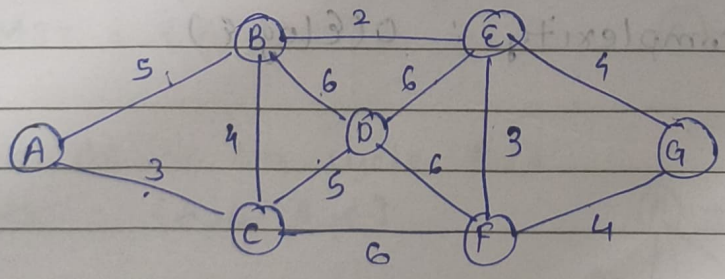
Select AD ... cycle found (Find returns false)
Discard.

Select AC ..
No cycle found

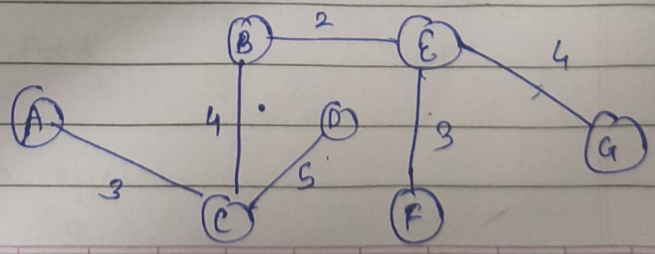


{ABCD}
∴ Minimum cost = 7

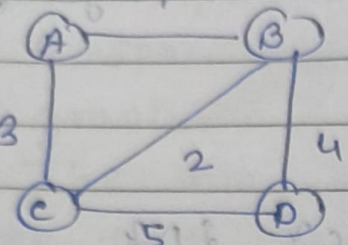
2)

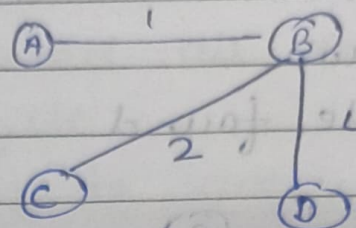


Minimum cost
= 3+4+5+2+3+4
= 21



3)





Minimum cost = 1+2+4 = 7

- Algorithm :

Input : Graph G(V, E)

Step 1 : Sort all edges in increasing order of weight

Step 2 : Make each vertex its own set (using union find)

Step 3 : For each edge (u,v) in sorted order :

if find(u) ≠ find(v)

Add edge to MST

union(u,v)

Step 4 : Stop when MST has V-1 edges

- Time complexity

Sorting edges → O(E log E)

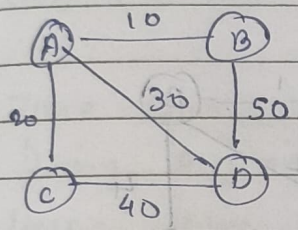
Union - Find operation → O(E α(V)) almost constant

Final time complexity : O(E log E)

- Prim's

1) Input graph
2) Select the starting vertex which is generally connected to the edge that has least weight
3) Find the least weight edge amongst the edges connected to the selected node.
4) If including the edge creates a cycle then reject it & keep repeating the process until all vertices are included in MST

1)



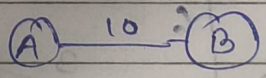
Step 1: Sort edges by weight

AB	10
AC	20
AD	30
CD	40
BD	50

Step 2: Apply Kruskal

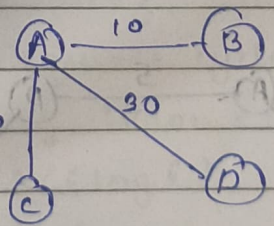
Pick A-B → No cycle Add

MST = {A-B}



Pick AC → No cycle Add

MST = {A-B, A-C}

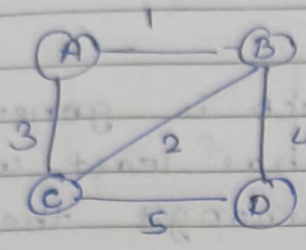


Pick AD → No cycle Add

CD and BD leads to cycle

∴ Minimum cost = 60

2)



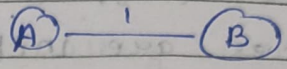
1) Sort edges

AB	1
BC	2
AC	3
BD	4
CD	5

2) Apply kruskal

Take AB → no cycle add

MST = {A - B}



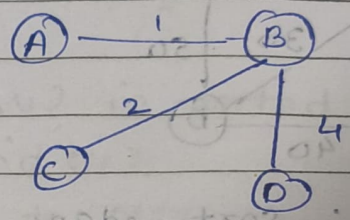
Take BC → no cycle add

MST = {A - B, B - C}

Take AC → cycle found

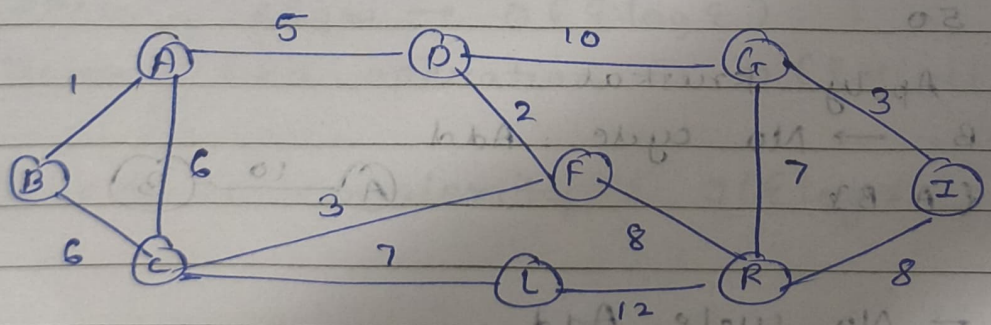
Take BD → no cycle add

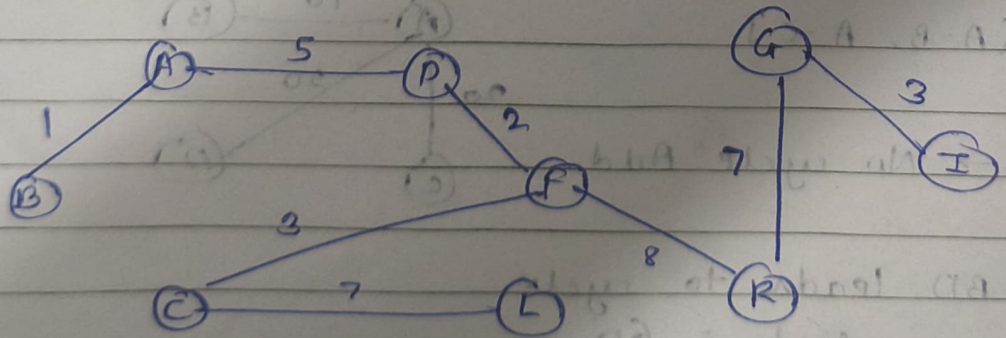
MST = {A - B, B - C, B - D}



Minimum cost = 1 + 2 + 4 = 7

3)





Minimum co

= 1 + 5 + 2 + 3 + 3

= 36

- Algorithm : Input : Graph G(V, E)
~~Step 1 : Sort all edges in increasing order of weight~~
~~Step 2 : Make each vertex its own set~~

Step 1 : Choose any starting vertex
Step 2 : Mark it as included in MST
Step 3 : Repeat until all vertices are included :
Select minimum weight edge
Connecting MST to a non-MST vertex
Step 4 : Add that edge to MST

- Time Complexity :
Using Adjacency Matrix : $O(V^2)$
Using Min Heap + Adjacency List : $O(E \log V)$

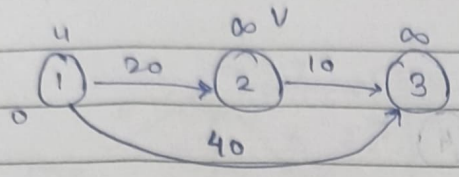
- Difference

Prim's	Kruskal
1) Vertex based approach	1) Edge-based approach
2) Starts from any one vertex	2) Starts by sorting all edges
3) Grows a single connected tree	3) Builds multiple tree (forest) and merges them
4) Does not require explicit cycle detection	4) Requires cycle detection using Union-Find
5) $O(V^2)$ or $O(E \log V)$	5) $O(E \log E)$
6) Best suited for dense graphs	6) Best suited for sparse graph.

- Single source shortest path (Dijkstra's algorithm):
Step 1) Mark all vertices as unvisited
2) Mark source vertex as zero and all as ∞.
3) Consider the source vertex as current vertex
4) Calculate the path length of neighbourhood vertices from current vertex.
5) If new path length is smaller than the previous path length than replace it with using relaxation technique
Relaxation technique:
for ∈ d.(u,v)
if
$d(v) > d(u) + w(u,v)$
then
$d(v) = d(u) + w(u,v)$;
6) Mark the current vertex as visited. Select the vertex with smallest path and continue till all vertices are marked as visited.

Dijkstra Algorithm is a greedy algorithm used to find the:
Shortest distance from single source vertex to all other vertices in weighted graph.
Condition:
Graph must have non-negative edge weights.

1)

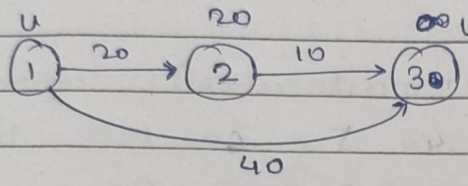


$d(v) > d(u) + w(u,v)$

$d(v) = d(u) + w(u,v)$

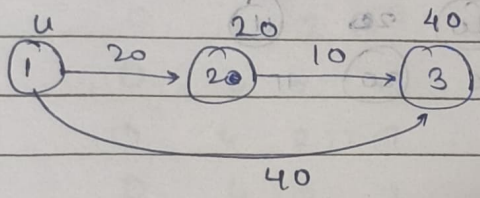
i) $d(u) = 0$, $w(u,v) = 20$ $\quad 20+0 = 20$

$d(v) = \infty$ $\quad \infty > 20$ $\quad \therefore d(v) = 20$

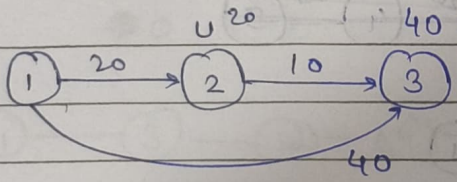


ii) $d(u) = 0$, $w(u,v) = 40$ $\quad 40+0 = 40$

$d(v) = \infty$ $\quad \infty > 40$ $\quad d(v) = 40$

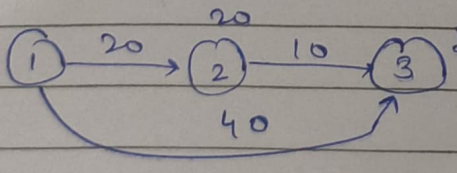


2)

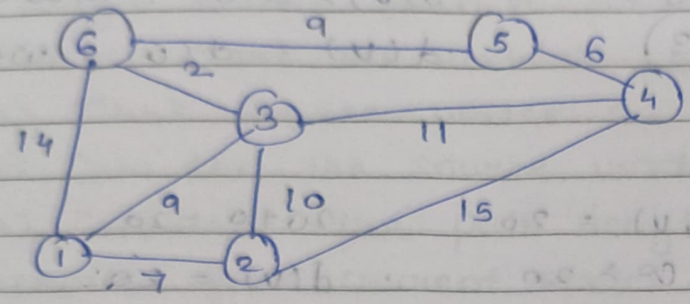


$d(u) = 20$, $w(u,v) = 10$ $\quad 20+10 = 30$

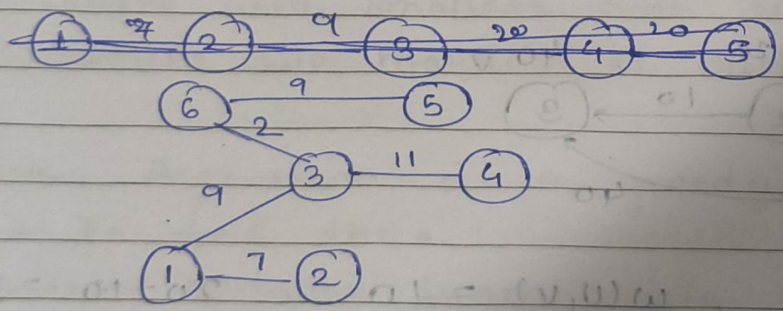
$d(v) = 40$ $\quad 40 > 30$ $\quad d(v) = 30$



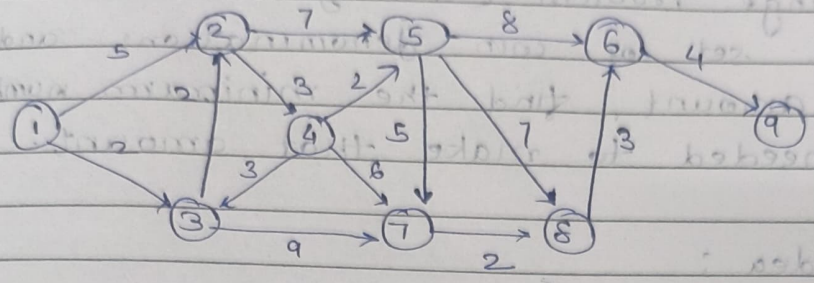
2)



Source	Destination				
1	2	3	4	5	6
	∞	∞	∞	∞	∞
1,2	7	9	∞	∞	14
1,2,3	7	9	22	∞	14
1,2,3,6	7	9	20	∞	11
1,2,3,6,4	7	9	20	20	11
1,2,3,6,4,5					



3)



~~Source~~ Destination

Visit	1	2	3	4	5	6	7	8	9
1	0	5	(2)	∞	∞	∞	∞	∞	∞
3	0	(4)	(2)	5	∞	∞	11	∞	∞
2	0	(4)	(2)	(7)	17	∞	11	∞	∞
4	0	(4)	(2)	(7)	(9)	∞	11	∞	∞
5	0	4	2	7	9	17	(14)	16	∞
7	0	4	2	7	9	19	14	(16)	∞
8	0	4	2	7	9	(19)	14	16	20
6	0	4	2	7	9	(19)	14	16	(23)
9	0	4	2	7	9	19	14	16	23

(1) — (3) — (2) — (4) — (5) — (7) — (8) — (9)

- Coin Change Problem (Greedy Method):
Given a set of coin denominations and a total amount, find the minimum number of coins needed to make that amount.

Greedy idea :
At each step pick largest coin
Subtract it from amount
Repeat till amount = 0

- Algorithm :
Input :
Coin denomination array C[]
Amount A
Step 1 : Sort coins in decreasing order.
Step 2 : For each coin C[i]
while A ≥ C[i] ;
A = A - C[i]
count ++
Step 3 : Continue until A = 0
Step 4 : Return count and Selected Coins

1) Coins :
1, 2, 5, 10, 20, 50
Amount : 93

Coins	Amount	Count
50	98 - 50 = 43	1
20	43 - 20 = 23	2
20	23 - 20 = 3	3
2	3 - 2 = 1	4
1	1 - 1 = 0	5

Final coins :

50 + 20 + 20 + 2 + 1

Total coins = 5

- Time complexity

Sorting → O(n log n)

Greedy selection → O(n)

Final complexity : O(n log n)